

Universidad de Carabobo
Facultad Experimental de Ciencias y Tecnología (FacYT)
Bárbula-Carabobo

Informe Práctica de Laboratorio 1

Bachiller:

Samuel Cordero 31.678.592

Dervis Martínez 31.456.326

Docente:

José Canache

Bárbula, 8 de julio del 2025

¿Cómo se implementa la recursividad en MIPS 32? ¿Qué papel cumple la pila (\$sp)?

En pocas palabras, la recursividad en MIPS 32 funciona apoyándose totalmente en la pila. Piensa en la pila como una pila de platos: cada vez que llamas a una función (o la función se llama a sí misma recursivamente), pones un "plato" con toda la información importante de esa llamada (como a dónde debe regresar el programa, qué valores estaba usando, etc.) encima de la pila.

¿Cómo se implementa?

Contexto de la llamada: Cada vez que se realiza una llamada a una función (incluyendo una llamada recursiva a sí misma), el procesador necesita guardar el "contexto" de la función que llama. Este contexto incluye:

- **Dirección de retorno (\$ra):** Es la dirección de la instrucción a la que el programa debe regresar una vez que la función llamada termina.
- **Registros salvados (\$s0 - \$s7):** Si la función llamada va a modificar el valor de estos registros, debe guardarlos en la pila antes de usarlos y restaurarlos antes de retornar, para que la función que la llamó encuentre sus valores originales.
- **Argumentos y variables locales:** Si hay más argumentos de los que pueden pasarse por los registros \$a0 - \$a3, o si hay variables locales que no caben en los registros, también se guardan en la pila.

Preparación de la llamada: Antes de saltar a la función recursiva:

- Los argumentos de la función se colocan en los registros de argumentos (\$a0 - \$a3) o, si no caben, se empujan a la pila.
- La dirección de retorno actual (valor de \$ra) se guarda en la pila, ya que la llamada recursiva modificará \$ra.

Cuerpo de la función recursiva:

- **Caso base:** La función debe tener un caso base que detenga la recursión y proporcione un valor de retorno sin hacer más llamadas a sí misma.
- **Paso recursivo:** Si no es el caso base, la función realiza cálculos y luego se llama a sí misma con argumentos modificados. Antes de esta llamada, se guarda el estado actual en la pila.

Restauración y retorno: Cuando la función recursiva llega a su caso base o termina una llamada recursiva:

- Se restauran los registros salvados y la dirección de retorno (\$ra) desde la pila.
- El valor de retorno se coloca en los registros de valores de retorno (\$v0 - \$v1).
- Se salta a la dirección de retorno almacenada en \$ra (instrucción jr \$ra).

¿Qué papel cumple la pila (\$sp)?

El registro \$sp (Stack Pointer) es fundamental en la implementación de la recursividad en MIPS. Cumple el papel de *puntero de la pila*, y siempre apunta a la parte superior de la pila (la dirección de memoria más baja). La pila en MIPS crece "hacia abajo", es decir, hacia direcciones de memoria más bajas.

El \$sp se utiliza para:

- **Asignar espacio en la pila:** Cuando una función se llama a sí misma o a otra función, se "empujan" (push) datos a la pila (como la dirección de retorno, registros que deben preservarse, y variables locales). Esto se hace decrementando el valor de \$sp (ya que la pila crece hacia abajo) y luego almacenando los datos en las direcciones de memoria apuntadas por \$sp o \$sp más un desplazamiento.

- **Liberar espacio de la pila:** Cuando una función termina, se "extraen" (pop) los datos de la pila. Esto se hace incrementando el valor de \$sp para liberar el espacio ocupado por el bloque de activación de la función.
- **Marco de pila (Stack Frame):** Cada vez que una función es invocada, se crea un "marco de pila" o "bloque de activación" en la pila. Este marco contiene toda la información necesaria para esa invocación específica de la función, como los parámetros de entrada, variables locales, y la dirección de retorno. \$sp ayuda a gestionar estos marcos.

Riesgos de Desbordamientos en la Pila (Stack Overflow)

El principal riesgo al usar recursividad en MIPS (o en cualquier arquitectura) es el *desbordamiento de la pila (Stack Overflow)*. Esto ocurre cuando la pila se queda sin espacio disponible en la memoria para almacenar los marcos de activación de las sucesivas llamadas a funciones.

Cada llamada recursiva añade un nuevo "marco" a la pila. Este marco contiene:

- Dirección de retorno: La dirección donde el programa debe continuar después de que la llamada actual termine.
- Registros guardados: Valores de registros que deben preservarse para la función que llamó.
- Argumentos de la función: Si son muchos o no caben en los registros \$a0-\$a3.
- Variables locales: Cualquier variable que sea local a esa instancia de la función.

Si la recursión es muy profunda (es decir, la función se llama a sí misma muchísimas veces antes de alcanzar el caso base), la pila puede crecer tanto que excede el límite de memoria asignado para ella, o incluso invadir otras secciones de memoria (como el segmento de datos o texto), causando un error crítico y el colapso del programa.

Cómo Mitigarlos:

- **Optimización del Código Recursivo:**
 - **Recursión de Cola (Tail Recursion Optimization):** Si la llamada recursiva es la última operación de la función, puedes reestructurarla para que no se cree un nuevo marco de pila, transformándola en un bucle eficiente. Esto es lo más potente para la recursión.
 - **Minimizar el Marco de Pila:** Reduce el número de registros guardados y la cantidad de datos locales en la pila por cada llamada.
- **Convertir a Iteración:**
 - Si es posible, reescribe el algoritmo usando bucles (iteración) en lugar de recursión.
 - Las soluciones iterativas no usan la pila para el estado de las llamadas y son inmunes al desbordamiento por profundidad.
- **Ajustar el Tamaño de la Pila (con precaución):**
 - En algunos entornos, se puede aumentar el tamaño máximo de la pila.
 - Sin embargo, esto es un parche temporal y no la solución ideal, ya que sigue habiendo un límite.
- **Revisión del Algoritmo:**
 - Asegúrate de que el caso base de tu recursión esté bien definido y se alcance siempre, para evitar recursiones infinitas.

Diferencias entre Secuencia de Fibonacci iterativo y recursivo

La forma en que calculamos la secuencia de Fibonacci, ya sea iterativa o recursivamente, marca una gran diferencia en cómo el programa usa la memoria y los registros del procesador.

Uso de Memoria (Pila)

El principal contraste aquí es la pila. Imagínate la pila como una torre de platos.

- El algoritmo iterativo es como construir la torre de uno en uno, usando un par de platos a la vez para el cálculo actual y el anterior. Una vez que un cálculo está hecho, no necesitamos ese plato en particular para los siguientes pasos, así que la torre de platos nunca crece mucho. Esto significa que apenas usa la pila de memoria y no tiene riesgo de desbordarse, sin importar cuán grande sea el número de Fibonacci que quieras calcular.
- El algoritmo recursivo, en cambio, es como si cada vez que necesitas un plato para un cálculo, pides dos platos nuevos para los cálculos que dependen de él, y así sucesivamente. Cada "pedido" apila más información (como a dónde debe regresar el programa, qué número se está procesando) en la torre. Para Fibonacci, esta torre puede crecer exponencialmente, lo que significa que, para números grandes, la pila puede quedarse sin espacio y provocar un desbordamiento (un "stack Overflow"), haciendo que el programa colapse.

Uso de Registros

Los registros son como pequeñas pizarras rápidas dentro del procesador para guardar valores temporales.

- El algoritmo iterativo usa principalmente registros temporales (\$t0, \$t1, etc.). Estos registros se encargan de guardar los dos últimos números de la secuencia y el contador del bucle. Como el cálculo avanza de forma lineal, los valores se actualizan directamente en estas pizarras, y no hay necesidad de copiarlos a otro lugar para mantener un seguimiento del "estado" anterior.
- El algoritmo recursivo es más complejo en su gestión de registros porque cada llamada a la función es independiente, pero necesita conocer el contexto de la que la llamó. Esto significa que el registro que lleva el número actual (\$a0) y el registro que dice a dónde regresar (\$ra) deben ser guardados en la pila antes de hacer una nueva llamada recursiva. Luego, se recuperan de la pila cuando la llamada recursiva termina. Este proceso de "guardar y restaurar" ocurre repetidamente, lo que añade más trabajo y consumo de pila en comparación con la versión iterativa.

En resumen, mientras que la recursión ofrece una forma elegante de expresar el problema de Fibonacci, la implementación iterativa es mucho más eficiente en el uso de los recursos de memoria y registros en MIPS, evitando los problemas de desbordamiento de pila para cálculos grandes.

Diferencias entre los ejemplos académicos del libro y un ejercicio completo y operativo en MIPS32

- **Naturaleza y Enfoque:** La teoría académica que encuentras en libros como el de Patterson y Hennessy se centra en conceptos fundamentales y principios de diseño. Te explica *qué* es la arquitectura MIPS, *por qué* se diseñó de cierta manera (como ser una arquitectura RISC), y cuáles son las reglas generales para que el procesador y el software interactúen (por ejemplo, las convenciones para usar la pila y los registros). Es un marco conceptual que te da las bases. En contraste, un ejercicio MIPS operativo es la aplicación concreta y práctica de esos principios. Es un programa real que *hace algo*, mostrando línea por línea *cómo* esas reglas se traducen en instrucciones para resolver una tarea específica. Aquí ves las decisiones exactas que un programador toma al escribir el código.
- **Alcance y Abstracción:** La teoría académica tiene un alcance muy amplio y es más abstracta. Cubre desde el diseño del conjunto de instrucciones (ISA) y cómo funcionan los pipelines, hasta el comportamiento de la memoria caché. Los ejemplos que ofrece el libro suelen ser fragmentos de código que ilustran un punto específico, como el mecanismo de una llamada a función o la gestión de un bucle, sin preocuparse por los detalles de un programa completo. Un ejercicio MIPS operativo, en cambio, es muy específico y detallado. Está enfocado en una única tarea. Además de las funciones o bucles, incluye toda la infraestructura necesaria para que el programa se compile y ejecute. Esto significa que también se ocupa de aspectos como las secciones de memoria donde se guardan datos (`.data`) y mensajes (`.asciiz`), el segmento donde reside el código (`.text`), y el punto de inicio desde donde el programa arranca (`main`).

- **Uso de la Pila y Registros:** En la teoría, se explican el modelo de la pila y las convenciones de los registros. El libro te dirá *por qué* la pila es esencial para las llamadas a funciones, cómo un "marco de pila" guarda la dirección de retorno (\$ra), argumentos y registros salvados (\$s). También define los roles de los registros (\$a para argumentos, \$v para retornos, \$t para temporales, etc.). En un ejercicio MIPS operativo, estas convenciones se implementan explícitamente. Ves cómo el puntero de pila (\$sp) se ajusta con instrucciones como `addi $sp, $sp, -X` para reservar espacio, y cómo se usan `lw` y `sw` para guardar y restaurar específicamente \$ra y otros registros o valores cruciales en la pila. También observas qué registros \$t se eligen para cálculos temporales o cómo un argumento como \$a0 podría necesitar ser salvado en la pila si su valor original es requerido después de una llamada recursiva, algo que la teoría explica como una posibilidad.
- **Manejo de Errores y Entradas/Salidas:** La teoría académica te proporciona los principios para entender los riesgos, como por qué la recursividad ineficiente puede causar un desbordamiento de pila o por qué ciertos algoritmos tienen un alto costo computacional. Te da las herramientas para identificar problemas conceptualmente. Un ejercicio operativo es donde estos riesgos se manifiestan directamente. Aquí puedes ver cómo un programa recursivo con muchas llamadas realmente agota la pila y colapsa. Además, los ejercicios operativos incorporan las llamadas al sistema (`syscalls`) exactas para interactuar con el usuario (mostrar mensajes, leer entradas), algo que la teoría describe en general, pero que es vital para un programa funcional.

Tutorial de la ejecución paso a paso en MARS

Para este tutorial, estamos asumiendo que ya tienes el código MIPS para la función de Fibonacci recursiva listo en MARS. Este es el que hace `fib(n-1) + fib(n-2)`, lo ejecutamos y vemos qué pasa "por dentro".

Paso 1: Preparar el terreno para MARS

Primero, necesitamos que MARS entienda nuestro código.

- **Carga tu Código:** Si no lo has hecho, pega tu código de Fibonacci recursivo en la ventana principal de MARS o ábrelo si lo tienes guardado como un `.asm` (o `.s`).
- **Ensamblar:** Busca el botón con forma de **martillo** (o ve a `Run > Assemble`). Dale clic. Si sale algo en la pestaña "Messages" de abajo que no sea "operation completed successfully", ¡es hora de corregir errores! MARS te dirá en qué línea está el problema. No puedes seguir hasta que esté limpio. Si todo bien, el panel central se llenará de números raros (código máquina) y el mensaje de "Messages" confirmará que todo salió bien.

Paso 2: Conocer Las Pestañas de MARS

Una vez ensamblado, MARS te muestra un montón de información. Las pestañas clave que vamos a vigilar son:

- **"Text Segment":** Aquí está nuestro código fuente, línea por línea, con su dirección de memoria al lado. Es donde vamos a seguir la "flechita" de ejecución.
- **"Registers":** Muestra el valor de cada uno de los 32 registros del MIPS en tiempo real. Aquí veremos cómo \$a0 (nuestro número de entrada), \$v0 (el resultado) y, muy importante, \$sp (el puntero de pila) y \$ra (la dirección de retorno) cambian.
- **"Stack":** Aquí es donde vamos a ver cómo la pila se llena y se vacía con cada llamada recursiva a Fibonacci. Veremos cómo se apilan los \$ra y los n que necesitamos guardar.
- **"Run I/O":** Esta es nuestra consola. Aquí nos pedirá el número para Fibonacci y aquí veremos el resultado final.

Paso 3: ¡A Depurar se ha Dicho! (Ejecución Paso a Paso)

Ahora sí, lo bueno. Vamos a "caminar" por el programa instrucción por instrucción.

- **Reinicia el Simulador:** Dale al botón de la **flecha verde curvada** (o Run > Reset). Esto pone todo "en cero" y nos asegura que el programa empieza desde el **main**. Fíjate que el PC (Program Counter) en la pestaña "Registers" debe apuntar a la primera instrucción de **main** (normalmente 0x00400000).
- **El Famoso Botón "Step":** Este es tu mejor amigo. Es el botón con un "1" y una flecha (o Run > Step). Cada vez que le das, MARS ejecuta *una única instrucción*. ¡Ni una más, ni una menos!
- Mientras le das a "Step", no le quites el ojo a:
 - **El PC (Program Counter):** En "Registers", verás cómo el PC salta a la siguiente instrucción que se va a ejecutar en el "Text Segment". Es como la flecha que te dice "ahora voy aquí".
 - **Los Registros:** ¡Obsérvalos con lupa! Después de cada **li**, **move**, **addi**, etc., verás cómo los valores en "Registers" cambian. Por ejemplo, cuando tu **main** te pide un número y lo lees, ese valor se mueve a \$a0.
 - **La Memoria:** Cuando llegues a las instrucciones **sw** (store word) o **lw** (load word), la pestaña "Stack" cobrará vida. En nuestro Fibonacci recursivo, esto es clave.

Paso 4: El Momento de la Recursión: JAL y la Pila en Acción

Aquí es donde la cosa se pone interesante con el Fibonacci recursivo:

- **Antes de la Primera Llamada a Fibonacci (en main):**
 - En **main**, cuando llegues a **jal Fibonacci**, mira el registro \$ra. Estará en cero o con un valor basura.
 - ¡Pum! Cuando ejecutes ese **jal** con "Step", verás cómo \$ra mágicamente se actualiza con la dirección de la instrucción *justo después* del **jal** en **main**. ¡Esa es la dirección a la que debemos regresar cuando Fibonacci termine!
- **Entrando en Fibonacci (¡Y la Pila Empieza a Crecer!):**
 - Lo primero que hace nuestra función Fibonacci es **addi \$sp, \$sp, -12**. Fíjate en el \$sp en "Registers": su valor disminuye (la pila crece hacia "abajo" en memoria). ¡Estamos reservando espacio para este "nivel" de la recursión!
 - Luego, **sw \$ra, 8(\$sp)** y **sw \$a0, 4(\$sp)**: Aquí es donde se guardan en la pila el \$ra (para saber a dónde volver *después* de esta llamada a Fibonacci) y el \$a0 (el número n de esta llamada, porque lo vamos a necesitar después para la suma $\text{fib}(n-1) + \text{fib}(n-2)$). ¡Verás estos valores aparecer en la pestaña "Stack"!
- **Las Llamadas Recursivas Internas (Fibonacci(n-1) y Fibonacci(n-2)):**
 - Cuando llegues a la primera **jal Fibonacci** dentro de la función Fibonacci (para n-1), ¡el proceso se repite!
 - Otro \$ra se guarda, otro \$a0 se guarda, y el \$sp vuelve a bajar.
 - Si le diste un número grande (como 5 o 6), vas a ver cómo la pestaña "Stack" se llena con múltiples pares de \$ra y \$a0, uno encima del otro. (Es la torre de llamadas recursivas que mencionamos).
 - Para $\text{fib}(n-2)$, sucederá lo mismo.
- **El Regreso a Casa (jr \$ra y la Pila se Vacía):**
 - Cuando una llamada a Fibonacci llega a un caso base ($n=0$ o $n=1$) o ya calculó su parte, verás cómo antes del **jr \$ra**, se ejecuta **lw \$ra, 8(\$sp)** y **addi \$sp, \$sp, 12**.
 - Observa cómo \$ra recupera la dirección de retorno que tenía justo antes de que esta llamada a Fibonacci empezara Y el \$sp vuelve a su posición anterior, liberando el espacio en la pila.
 - **jr \$ra** te devolverá a la instrucción donde dejaste la llamada anterior.
 - Y así, la pila se va vaciando de arriba hacia abajo, "desenrollando" las llamadas recursivas.

Justificación de la Elección del Enfoque Recursivo de Fibonacci en MIPS:

Bueno, profe, si tuviera que justificar por qué elegimos el Fibonacci recursivo para un trabajo en MIPS, más allá de que el enunciado lo pida, tendría que enfocarme en la claridad conceptual para quien lo lee y, quizás, en un contexto académico muy específico donde la eficiencia no sea la prioridad principal. (ya veremos por qué).

Claridad y Correspondencia Directa con la Definición Matemática

- **La Mayor Fortaleza:** La principal ventaja del Fibonacci recursivo es su claridad y la correspondencia directa con la definición matemática. La secuencia de Fibonacci se define recursivamente como $F(n) = F(n-1) + F(n-2)$, con casos base para $F(0) = 0$ y $F(1) = 1$.
- **Facilidad de Comprensión (para Humanos):** El código MIPS que escribimos para el enfoque recursivo es casi una traducción uno a uno de esa definición. Es muy fácil para un programador que entiende la recursión en alto nivel ver inmediatamente lo que hace el código en ensamblador. No hay que "desenredar" el bucle y la actualización de las dos variables anteriores. Esto lo hace altamente legible para fines educativos o de demostración del concepto.

Demostración de Conceptos Avanzados de MIPS y Arquitectura

- **Manejo de la Pila:** Si el objetivo del ejercicio es ilustrar el uso de la pila y las convenciones de llamada a procedimiento (jal, \$ra, \$sp, guardado de \$a) en un escenario de complejidad creciente, el Fibonacci recursivo es un ejemplo académico perfecto. Permite ver de forma dramática cómo la pila crece y decrece, cómo se guardan y restauran los registros en cada llamada, y cómo se gestiona el contexto de ejecución. Es un "laboratorio" excelente para entender a fondo la memoria y las llamadas a funciones.
- **Recursión en Ensamblador:** Sirve para demostrar que la recursión es posible y cómo se implementa a bajo nivel, incluso sin el soporte directo del compilador para el "azúcar sintáctico" que tendríamos en C o Java.

Limitaciones Conocidas (Y Por Qué A Veces Se Aceptan en lo Académico)

- **Eficiencia Comprometida:** En la práctica real, este algoritmo es terriblemente ineficiente. Si el valor de n fuera a ser más grande que unos 10-15 (dependiendo de la pila disponible), el programa colapsaría. No es para un uso productivo.
- **Propósito Educativo:** Sin embargo, en un entorno universitario, a veces se usan ejemplos "ineficientes" precisamente para ilustrar los problemas de rendimiento y los costos de los recursos (como la memoria de la pila) cuando un enfoque no es el más adecuado. Es una herramienta pedagógica para mostrar "lo que no se debe hacer" o "lo que cuesta hacer esto de esta manera". Es decir, se elige no por su eficiencia, sino por su capacidad de enseñar una lección sobre eficiencia y uso de recursos.

Así que, profe, si el trabajo pide una justificación de la recursividad, diría que se elige por su claridad inigualable para representar la definición matemática y por su valor como herramienta didáctica para comprender profundamente el funcionamiento de la pila y las llamadas a funciones en MIPS. Sería para un ejercicio que busca la comprensión del concepto de recursión a bajo nivel y el manejo de la memoria, más que para un programa que vaya a resolver problemas de Fibonacci en producción.

Análisis y discusión de Resultados

Nuestra exploración de los algoritmos de Fibonacci en MIPS 32, desde su implementación recursiva hasta la iterativa, y nuestra experiencia en el simulador MARS bajo la lente de libros como el de Patterson y Hennessy, nos ha dado una visión clara de cómo la teoría de la arquitectura se traduce en la práctica del ensamblador.

- **La Recursividad en MIPS: Un Viaje a la Pila:** Empezamos viendo cómo la recursividad, un concepto elegante en programación, se "aterriza" en MIPS. Descubrimos que implica el uso intensivo de la *pila de memoria*: cada llamada a la función (`jal`) significa guardar la dirección de retorno (`$ra`) y los argumentos necesarios (`$a0`) en la pila (`sw $ra, X($sp), sw $a0, Y($sp)`). Esto nos permitió entender la mecánica de cómo MIPS "recuerda" a dónde volver y qué valores necesita cada instancia de la función, viendo en MARS cómo el `$sp` baja y la pila se llena.
- **Dos Caminos para Fibonacci: Eficiencia vs. Claridad:**
 - El *Fibonacci recursivo*, fiel a su definición matemática, es *directo y claro* de entender para nosotros. Es un ejemplo perfecto para *ilustrar cómo funciona la pila* con múltiples llamadas anidadas, mostrándonos el "desenrollado" de la recursión. Sin embargo, su ejecución en MARS reveló su *gran debilidad*: una explosión de llamadas y un *crecimiento exponencial de la pila*, lo que lleva rápidamente a un temido *desbordamiento (stack overflow)* y a un despilfarro de ciclos de CPU por recalcular lo mismo una y otra vez.
 - El *Fibonacci iterativo*, por otro lado, demostró ser el campeón de la *eficiencia*. Al usar un bucle simple y manejar los valores anteriores con unos pocos *registros temporales (\$t)*, evita la dependencia de la pila para las llamadas anidadas. Esto se traduce en un *uso constante y mínimo de memoria* y un *rendimiento lineal ($O(N)$)*, haciendo que pueda calcular números mucho más grandes sin problemas.
- **Del Libro a la Línea de Código:** La comparación con la teoría de Patterson y Hennessy fue clave. Sus principios sobre la arquitectura RISC, el manejo de registros y las convenciones de la pila cobraron vida en nuestros códigos. Vimos cómo los `jal` y `jr $ra` son la implementación de las llamadas a procedimiento, cómo los registros `$a`, `$v`, `$t`, `$s` cumplen sus roles, y cómo las secciones `.data`, `.text` y el `main` son esenciales para un programa *operativo* completo, a diferencia de los ejemplos a veces fragmentados de los libros. MARS, con su ejecución paso a paso, fue nuestra "ventana" para observar estos principios en acción, desde el ajuste del `$sp` hasta el flujo de control.